COMMENT CONSTRUIRE, EVALUER ET SELECTIONNER EFFICACEMENT UN MODELE DE MACHINE LEARNING ?

Objectifs - Etapes

- **Analyse exploratoire des données** ;
- **Gestion des valeurs manquantes** ;
- **Gestion des valeurs aberrantes (*outliers*)** ;
- **Autres étapes possibles de prétraitement des données** ;
- **Division de l'ensemble des données (données d'entraînement, de validation et de test)** ;
- **Construction de plusieurs modèles à partir de différents algorithmes de Machine Learning** ;
- **Comparaison des performances des algorithmes** ;
- **Sélection et Evaluation du modèle final**.

Importation des librairies et des données

```
# Librairies et fonctions

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.get_dataset_names()
```

```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic']
```

```
df = sns.load_dataset('titanic')
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Étapes suivantes : Générer du code avec df New interactive sheet

```
df = df.drop(['alive', 'who', 'embarked', 'class', 'deck'], axis = 1)
df.head(3)
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	embark_town	alone
0	0	3	male	22.0	1	0	7.2500	True	Southampton	False
1	1	1	female	38.0	1	0	71.2833	False	Cherbourg	False
2	1	3	female	26.0	0	0	7.9250	False	Southampton	True

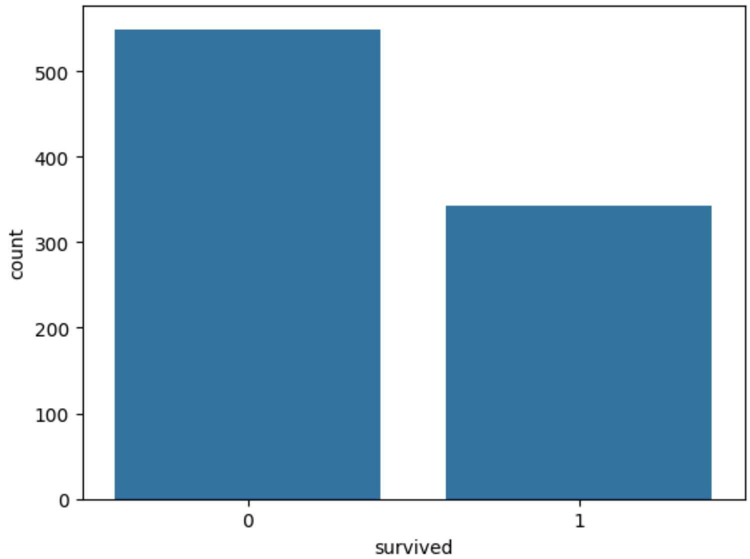
Étapes suivantes : Générer du code avec df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   survived     891 non-null    int64  
 1   pclass       891 non-null    int64  
 2   sex          891 non-null    object 
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64  
 5   parch        891 non-null    int64  
 6   fare         891 non-null    float64
 7   adult_male   891 non-null    bool   
 8   embark_town  889 non-null    object 
 9   alone        891 non-null    bool   
dtypes: bool(2), float64(2), int64(4), object(2)
memory usage: 57.6+ KB
```

Visualisation des données

```
sns.countplot(x = 'survived', data=df);
```

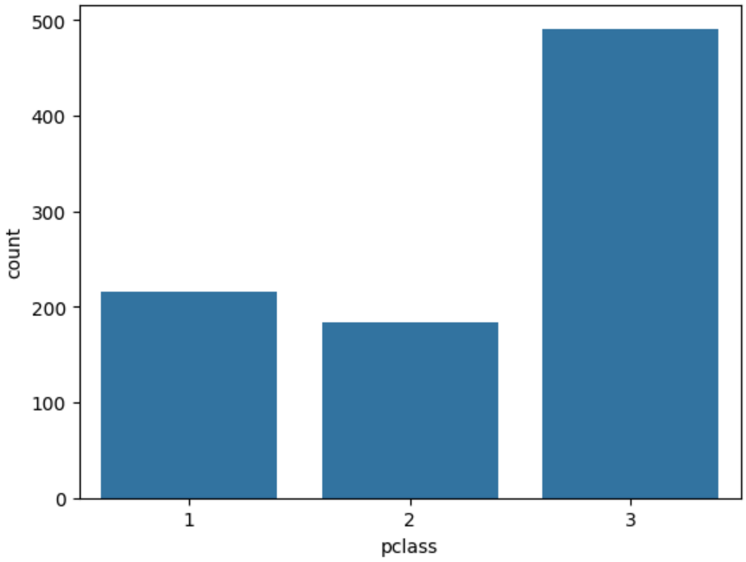



```
df['survived'].value_counts(normalize = True)
```

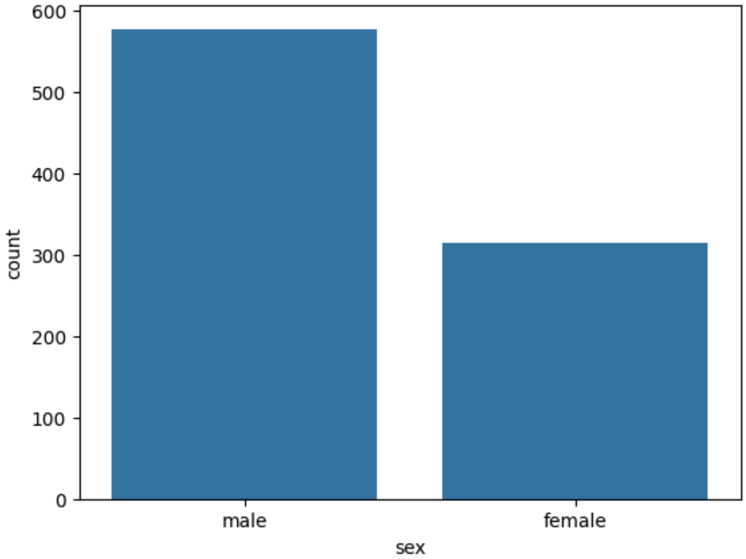
survived	proportion
0	0.616162
1	0.383838

dtype: float64

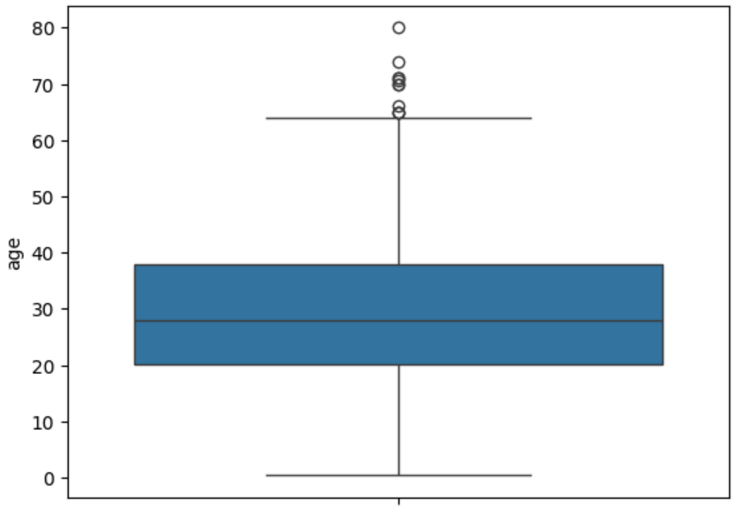
```
sns.countplot(x = 'pclass', data=df);
```




```
sns.countplot(x = 'sex', data=df);
```

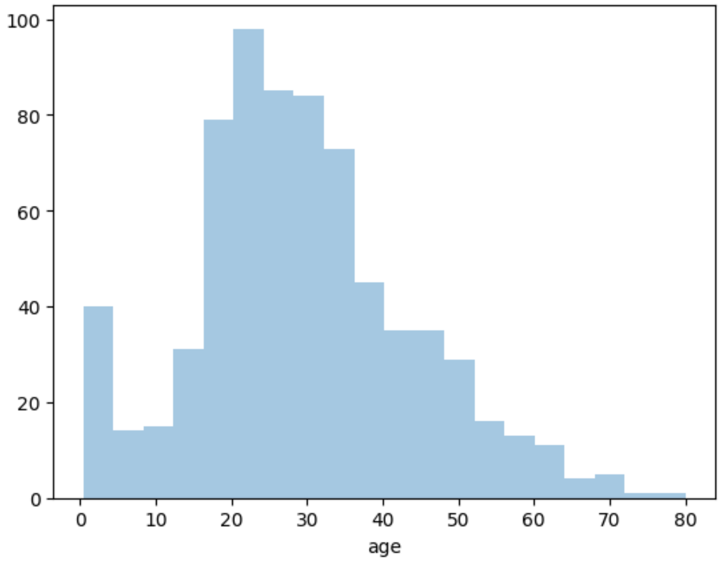



```
sns.boxplot(y = 'age', data = df);
```

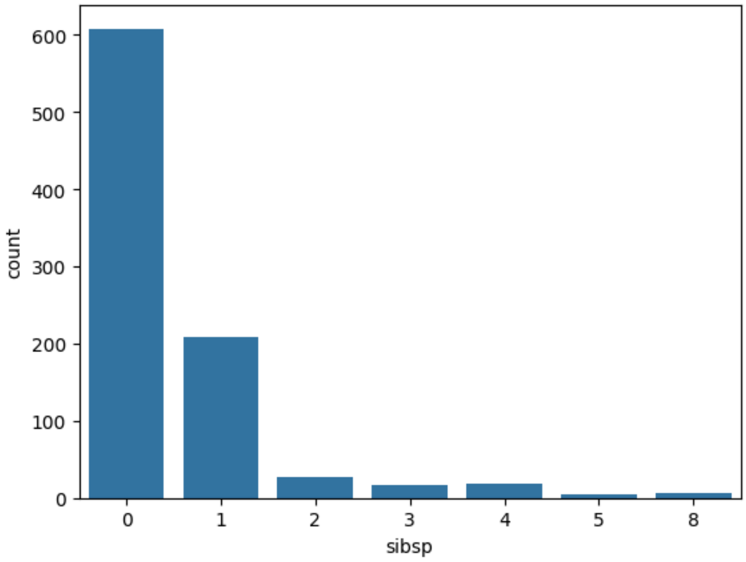



```
sns.distplot(df['age'], kde=False, bins=20);
```

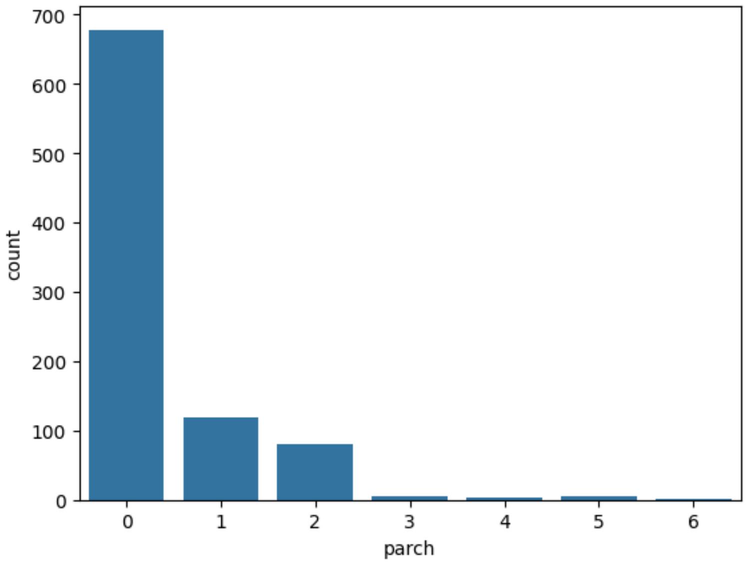
```
/tmp/ipython-input-2995001087.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df['age'], kde=False, bins=20);
```



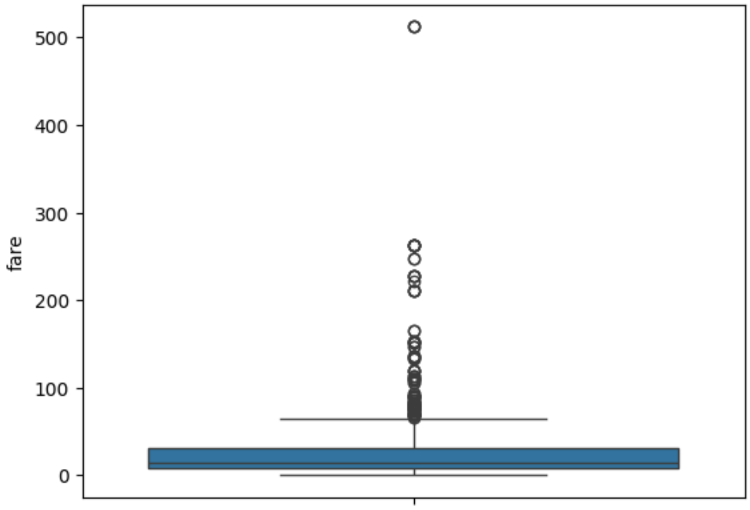

```
sns.countplot(x = 'sibsp', data=df);
```




```
sns.countplot(x = 'parch', data=df);
```

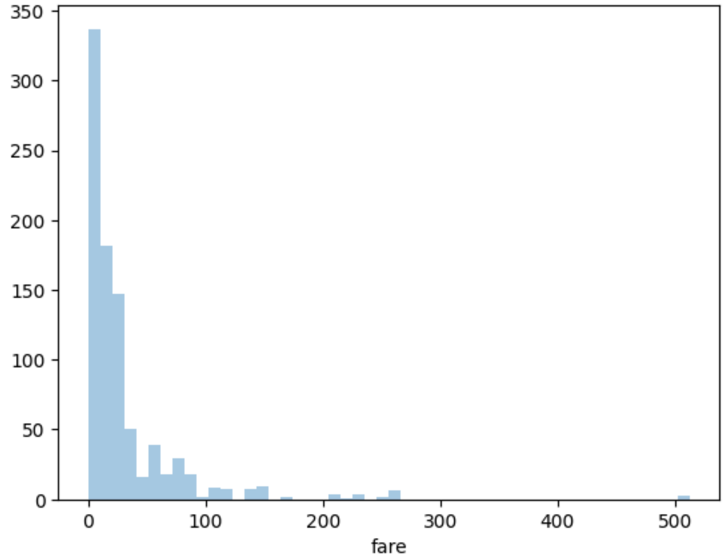



```
sns.boxplot(y = 'fare', data = df);
```

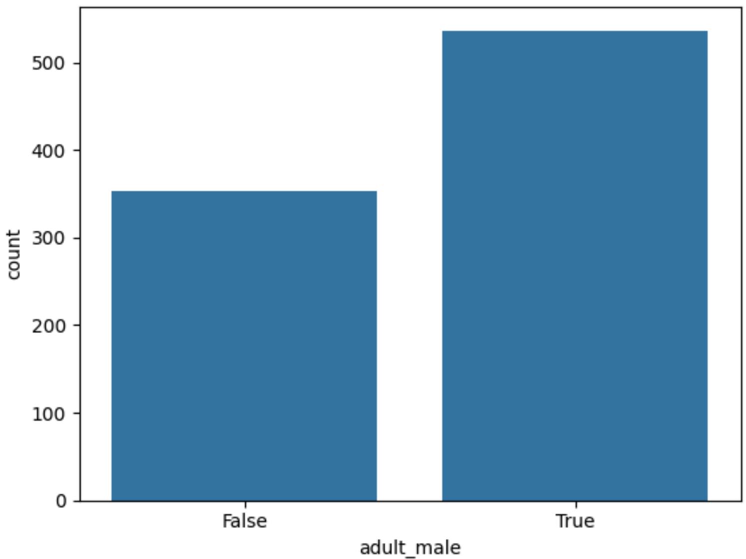



```
sns.distplot(df['fare'], kde=False);
```

```
/tmp/ipython-input-3649619019.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df['fare'], kde=False);
```



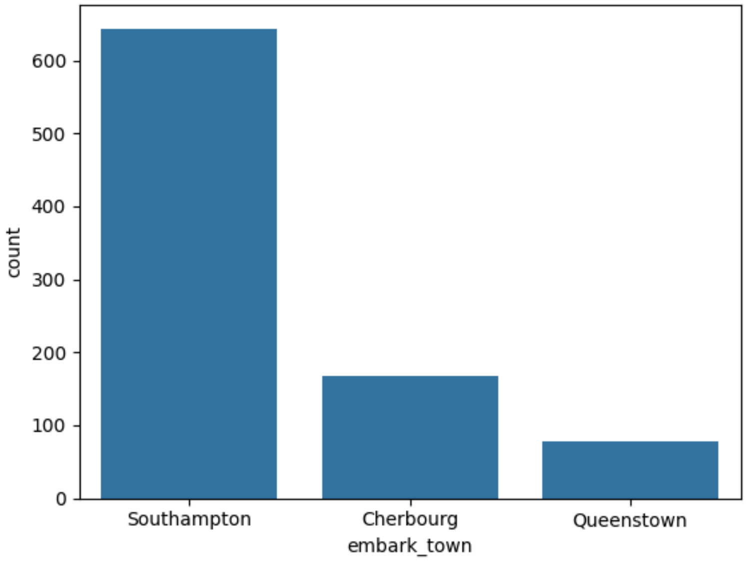

```
sns.countplot(x = 'adult_male', data=df);
```



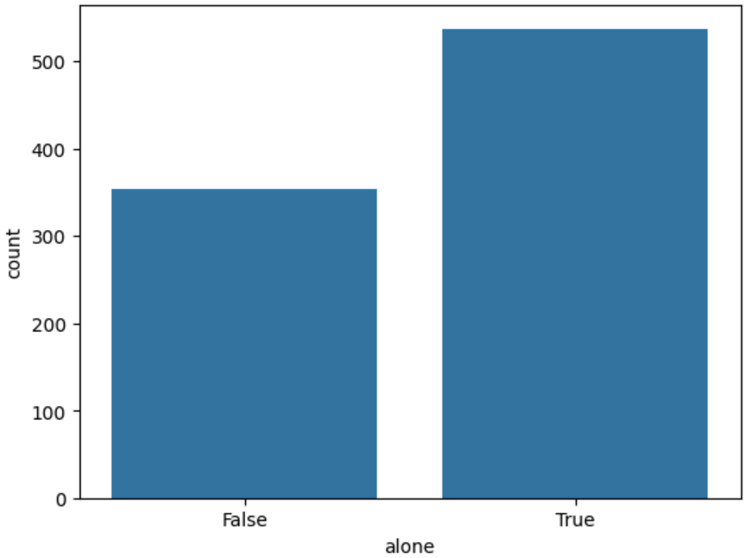

```
pd.crosstab(df['sex'], df['adult_male'])
```

adult_male	False	True
sex		
female	314	0
male	40	537

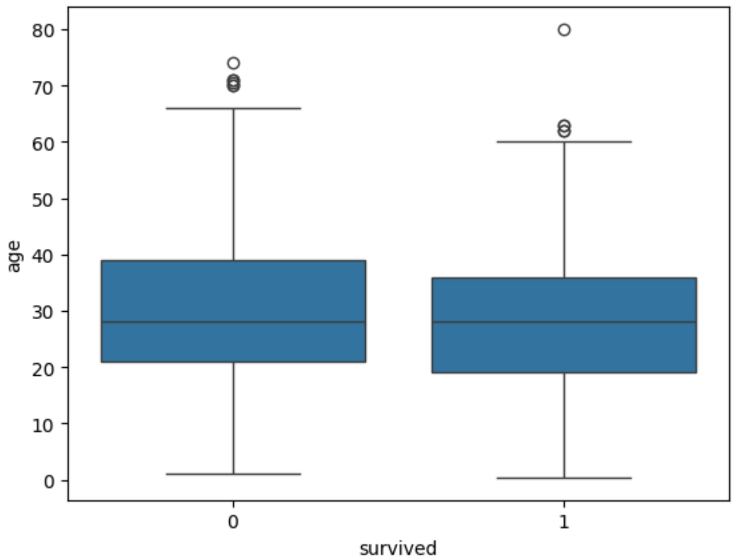
```
sns.countplot(x = 'embark_town', data=df);
```



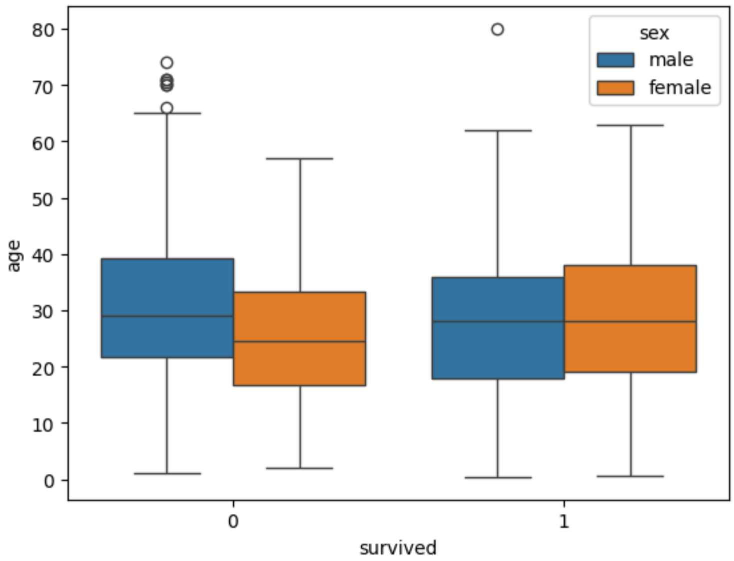

```
sns.countplot(x = 'alone', data=df);
```



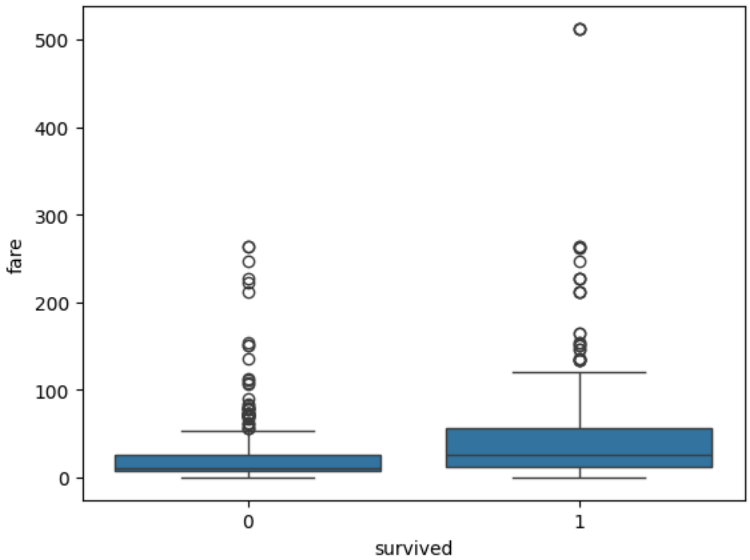

```
# Relation entre âge et variable cible

sns.boxplot(x = 'survived', y = 'age', data=df);
```



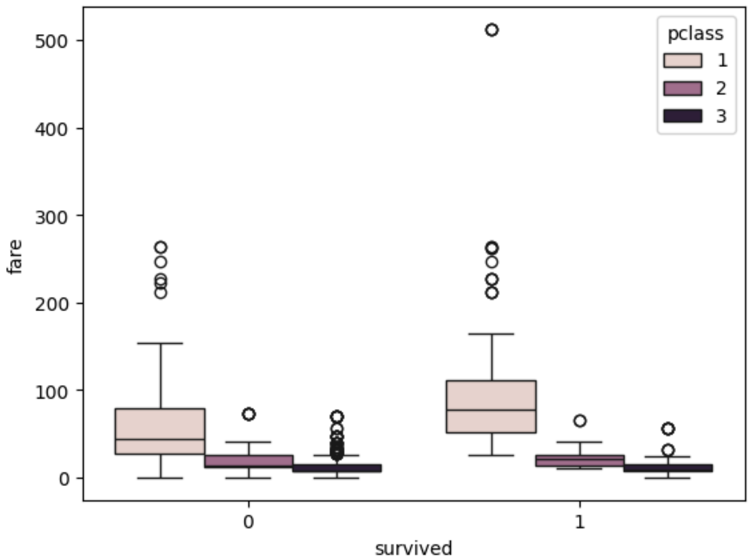

```
sns.boxplot(x = 'survived', y = 'age', hue = 'sex', data=df);
```



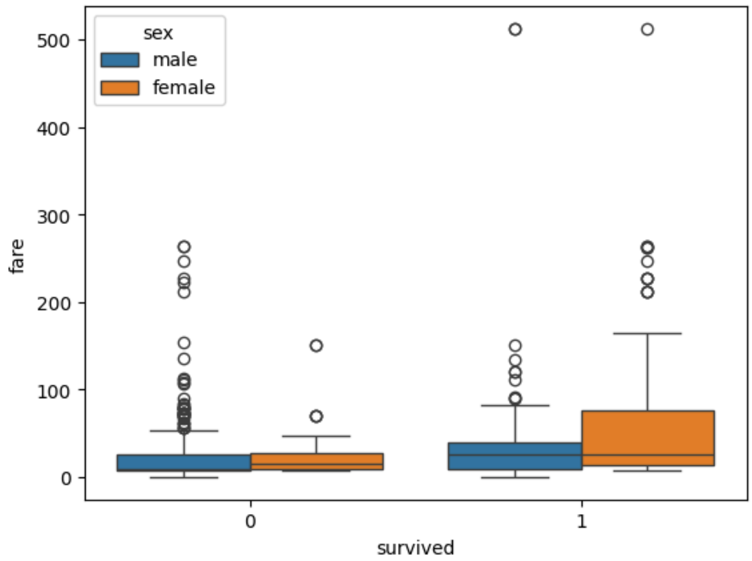

```
sns.boxplot(x = 'survived', y = 'fare', data = df);
```




```
sns.boxplot(x = 'survived', y = 'fare', hue = 'pclass', data = df);
```




```
sns.boxplot(x = 'survived', y = 'fare', hue = 'sex', data = df);
```




```
df.describe()
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
df['fare'].quantile(0.99)
```

```
np.float64(249.00622000000035)
```

Nettoyage des données

```
df.isna().sum()
```

	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
adult_male	0
embark_town	2
alone	0

dtype: int64

```
df.fillna(value={'age':df['age'].median()}, inplace=True)
```

```
df['age'].std()
```

```
13.019696550973194
```

```
df['embark_town'].value_counts(normalize=True)
```

embark_town	proportion
Southampton	0.724409
Cherbourg	0.188976
Queenstown	0.086614

dtype: float64

```
df.fillna(value={'embark_town':'Southampton'}, inplace=True)
```

```
df.isna().sum()
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
adult_male	0
embark_town	0
alone	0

dtype: int64

```
# valeur < Q1 - 1,5*IQR ou valeur > Q3 + 1,5*IQR

def finding_outliers(data, variable_name):
  iqr = data[variable_name].quantile(0.75) - data[variable_name].quantile(0.25)
  lower = data[variable_name].quantile(0.25) - 1.5 * iqr
  upper = data[variable_name].quantile(0.75) + 1.5 * iqr
  return data[(data[variable_name] < lower) | (data[variable_name] > upper)]
```

```
finding_outliers(df, 'fare').sort_values('fare')
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	embark_town	alone
151	1	1	female	22.0	1	0	66.6000	False	Southampton	False
336	0	1	male	29.0	1	0	66.6000	True	Southampton	False
369	1	1	female	24.0	0	0	69.3000	False	Cherbourg	True
641	1	1	female	24.0	0	0	69.3000	False	Cherbourg	True
201	0	3	male	28.0	8	2	69.5500	True	Southampton	False
...	...	...	...	...	...	...	...	...	...	...
341	1	1	female	24.0	3	2	263.0000	False	Southampton	False
438	0	1	male	64.0	1	4	263.0000	True	Southampton	False
258	1	1	female	35.0	0	0	512.3292	False	Cherbourg	True
679	1	1	male	36.0	0	1	512.3292	True	Cherbourg	False
737	1	1	male	35.0	0	0	512.3292	True	Cherbourg	True

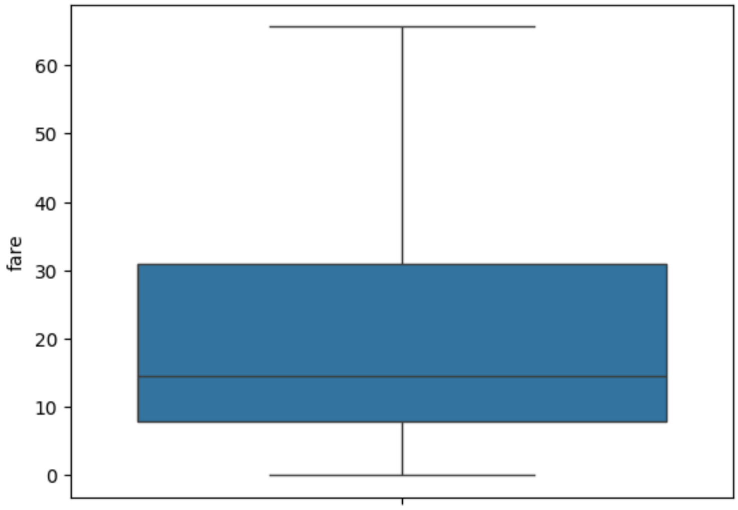
116 rows × 10 columns

```
iqr_fare = df['fare'].quantile(0.75) - df['fare'].quantile(0.25)
float(df['fare'].quantile(0.75) + 1.5* iqr_fare)
```

```
65.6344
```

```
df.loc[(finding_outliers(df, 'fare').index, 'fare')] = df['fare'].quantile(0.75) + 1.5 * iqr_fare
```

```
sns.boxplot(y = 'fare', data=df);
```




```
float(df['age'].quantile(0.25) - 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25)))
```

```
2.5
```

```
float(df['age'].quantile(0.75) + 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25)))
```

```
54.5
```

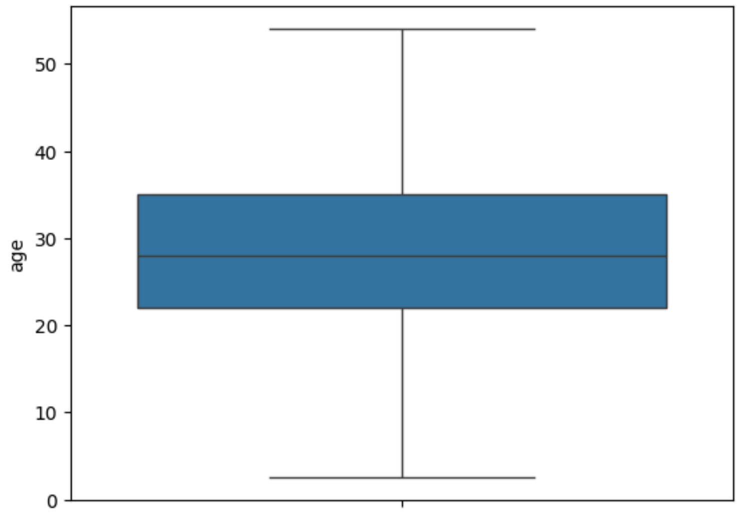
```
finding_outliers(df, 'age').sort_values('age')
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	embark_town	alone
803	1	3	male	0.42	0	1	8.5167	False	Cherbourg	False
755	1	2	male	0.67	1	1	14.5000	False	Southampton	False
469	1	3	female	0.75	2	1	19.2583	False	Cherbourg	False
644	1	3	female	0.75	2	1	19.2583	False	Cherbourg	False
831	1	2	male	0.83	1	1	18.7500	False	Southampton	False
...	...	...	...	...	...	...	...	...	...	...
116	0	3	male	70.50	0	0	7.7500	True	Queenstown	True
96	0	1	male	71.00	0	0	34.6542	True	Cherbourg	True
493	0	1	male	71.00	0	0	49.5042	True	Cherbourg	True
851	0	3	male	74.00	0	0	7.7750	True	Southampton	True
630	1	1	male	80.00	0	0	30.0000	True	Southampton	True

66 rows × 10 columns

```
df.loc[df['age'] < df['age'].quantile(0.25) - 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25)),
       'age'] = df['age'].quantile(0.25) - 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25))

df.loc[df['age'] > df['age'].quantile(0.75) + 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25)),
       'age'] = df['age'].quantile(0.55) + 1.5 * (df['age'].quantile(0.75) - df['age'].quantile(0.25))
```

```
sns.boxplot(y = 'age', data=df);
```




```
df.head(3)
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	embark_town	alone
0	0	3	male	22.0	1	0	7.2500	True	Southampton	False
1	1	1	female	38.0	1	0	65.6344	False	Cherbourg	False
2	1	3	female	26.0	0	0	7.9250	False	Southampton	True

Étapes suivantes : Générer du code avec df New interactive sheet

Préparation des données

```
df['sex'].replace({'female':0, 'male':1}, inplace = True)
```

```
/tmp/ipython-input-1649213356.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['sex'].replace({'female':0, 'male':1}, inplace = True)
/tmp/ipython-input-1649213356.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  df['sex'].replace({'female':0, 'male':1}, inplace = True)
```

```
df['alone'] = df['alone'].astype('int')
df['adult_male'] = df['adult_male'].astype('int')
```

```
df['embark_town'].unique()
```

```
array(['Southampton', 'Cherbourg', 'Queenstown'], dtype=object)
```

```
embark_dummies = pd.get_dummies(df['embark_town'], drop_first=True)
embark_dummies
```

	Queenstown	Southampton
0	False	True
1	False	False
2	False	True
3	False	True
4	False	True
...	...	...
886	False	True
887	False	True
888	False	True
889	False	False
890	True	False

891 rows × 2 columns

Étapes suivantes : Générer du code avec `embark_dummies` New interactive sheet

```
df = pd.concat([df, embark_dummies], axis=1)
df.head(3)
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	embark_town	alone	Queenstown	Southampton	Queenstown	So
0	0	3	1	22.0	1	0	7.2500	1	Southampton	0	False	True	False	
1	1	1	0	38.0	1	0	65.6344	0	Cherbourg	0	False	False	False	
2	1	3	0	26.0	0	0	7.9250	0	Southampton	1	False	True	False	

Étapes suivantes : Générer du code avec `df` New interactive sheet

```
del df['embark_town']
```

```
df.head(3)
```

	survived	pclass	sex	age	sibsp	parch	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton
0	0	3	1	22.0	1	0	7.2500	1	0	False	True	False	True
1	1	1	0	38.0	1	0	65.6344	0	0	False	False	False	False
2	1	3	0	26.0	0	0	7.9250	0	1	False	True	False	True

Étapes suivantes : Générer du code avec `df` New interactive sheet

```
df['family'] = df['sibsp'] + df['parch']

df.drop(['sibsp', 'parch'], axis=1, inplace=True)

df.head(3)
```

	survived	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family
0	0	3	1	22.0	7.2500	1	0	False	True	False	True	1
1	1	1	0	38.0	65.6344	0	0	False	False	False	False	1
2	1	3	0	26.0	7.9250	0	1	False	True	False	True	0

Étapes suivantes : Générer du code avec `df` New interactive sheet

```
# Division des données 60% train, 20% validation, 20% test

from sklearn.model_selection import train_test_split
seed = 111
X = df.drop('survived', axis=1)
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.4,
                                                    random_state = seed, stratify=y)
```

```
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size = 0.5,
                                                random_state = seed, stratify=y_test)
```

```
y.value_counts(normalize=True)
```

survived	proportion
0	0.616162
1	0.383838

dtype: float64

```
y_train.value_counts(normalize=True)
```

survived	proportion
0	0.616105
1	0.383895

dtype: float64

```
y_val.value_counts(normalize=True)
```

survived	proportion
0	0.617978
1	0.382022

dtype: float64

```
y_test.value_counts(normalize=True)
```

survived	proportion
0	0.614525
1	0.385475

dtype: float64

```
# Normalisation
```

```
# Méthode de sur-échantillonnage (upsampling)

from sklearn.utils import resample

X2 = X_train
X2['survived'] = y_train.values
X2.head(3)
```

	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family	survived
692	3	1	28.0	56.4958	1	1	False	True	False	True	0	1
150	2	1	51.0	12.5250	1	1	False	True	False	True	0	0
886	2	1	27.0	13.0000	1	1	False	True	False	True	0	0

Étapes suivantes : Générer du code avec X2 New interactive sheet

```
minority = X2[X2.survived == 1]
majority = X2[X2.survived == 0]

minority_upsampled = resample(minority, replace=True, n_samples = len(majority))

minority_upsampled
```

	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family	survived
448	3	0	5.0	19.2583	0	0	False	False	False	False	3	1
393	1	0	23.0	65.6344	0	0	False	False	False	False	1	1
279	3	0	35.0	20.2500	0	0	False	True	False	True	2	1
549	2	1	8.0	36.7500	0	0	False	True	False	True	2	1
146	3	1	27.0	7.7958	1	1	False	True	False	True	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...
690	1	1	31.0	57.0000	1	0	False	True	False	True	1	1
348	3	1	3.0	15.9000	0	0	False	True	False	True	2	1
740	1	1	28.0	30.0000	1	1	False	True	False	True	0	1
829	1	0	47.5	65.6344	0	1	False	True	False	True	0	1
571	1	0	53.0	51.4792	0	0	False	True	False	True	2	1

329 rows × 12 columns

Étapes suivantes : Générer du code avec `minority_upsampled` New interactive sheet

```
majority.shape
```

```
(329, 12)
```

```
upsampled = pd.concat([majority, minority_upsampled])
upsampled
```

	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family	survived
150	2	1	51.0	12.5250	1	1	False	True	False	True	0	0
886	2	1	27.0	13.0000	1	1	False	True	False	True	0	0
149	2	1	42.0	13.0000	1	1	False	True	False	True	0	0
249	2	1	54.0	26.0000	1	0	False	True	False	True	1	0
464	3	1	28.0	8.0500	1	1	False	True	False	True	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
690	1	1	31.0	57.0000	1	0	False	True	False	True	1	1
348	3	1	3.0	15.9000	0	0	False	True	False	True	2	1
740	1	1	28.0	30.0000	1	1	False	True	False	True	0	1
829	1	0	47.5	65.6344	0	1	False	True	False	True	0	1
571	1	0	53.0	51.4792	0	0	False	True	False	True	2	1

658 rows × 12 columns

Étapes suivantes : Générer du code avec `upsampled` New interactive sheet

```
upsampled['survived'].value_counts(normalize = True)
```

	proportion
survived	
0	0.5
1	0.5

dtype: float64

```
X_train_up = upsampled.drop('survived', axis=1)
y_train_up = upsampled['survived']
```

```
# Méthode de sous-échantillonnage

majority_downsampled = resample(majority, replace=False, n_samples = len(minority))

majority_downsampled
```

	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family	survived
773	3	1	28.0	7.2250	1	1	False	False	False	False	0	0
465	3	1	38.0	7.0500	1	1	False	True	False	True	0	0
229	3	0	28.0	25.4667	0	0	False	True	False	True	4	0
586	2	1	47.0	15.0000	1	1	False	True	False	True	0	0
836	3	1	21.0	8.6625	1	1	False	True	False	True	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
661	3	1	40.0	7.2250	1	1	False	False	False	False	0	0
760	3	1	28.0	14.5000	1	1	False	True	False	True	0	0
528	3	1	39.0	7.9250	1	1	False	True	False	True	0	0
24	3	0	8.0	21.0750	0	0	False	True	False	True	4	0
41	2	0	27.0	21.0000	0	0	False	True	False	True	1	0

205 rows × 12 columns

Étapes suivantes : Générer du code avec `majority_downsampled` New interactive sheet

```
downsampled = pd.concat([minority, majority_downsampled])
downsampled
```

	pclass	sex	age	fare	adult_male	alone	Queenstown	Southampton	Queenstown	Southampton	family	survived
692	3	1	28.0	56.4958	1	1	False	True	False	True	0	1
151	1	0	22.0	65.6344	0	0	False	True	False	True	1	1
788	3	1	2.5	20.5750	0	0	False	True	False	True	3	1
609	1	0	40.0	65.6344	0	1	False	True	False	True	0	1
697	3	0	28.0	7.7333	0	1	True	False	True	False	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...
661	3	1	40.0	7.2250	1	1	False	False	False	False	0	0
760	3	1	28.0	14.5000	1	1	False	True	False	True	0	0
528	3	1	39.0	7.9250	1	1	False	True	False	True	0	0
24	3	0	8.0	21.0750	0	0	False	True	False	True	4	0
41	2	0	27.0	21.0000	0	0	False	True	False	True	1	0

410 rows × 12 columns

Étapes suivantes : Générer du code avec `downsampled` New interactive sheet

```
X_train_down = downsampled.drop(['survived'], axis=1)
y_train_down = downsampled['survived']
```

Modélisation

```
# Sélection de variables importantes

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

rf = RandomForestClassifier(random_state = seed)

rf.fit(X_train_up, y_train_up)

accuracy_score(y_val, rf.predict(X_val))
```

```
0.7303370786516854
```

```
print(X_train_up.columns)
print(rf.feature_importances_)
```

```
Index(['pclass', 'sex', 'age', 'fare', 'adult_male', 'alone', 'Queenstown',
       'Southampton', 'Queenstown', 'Southampton', 'family'],
      dtype='object')
```

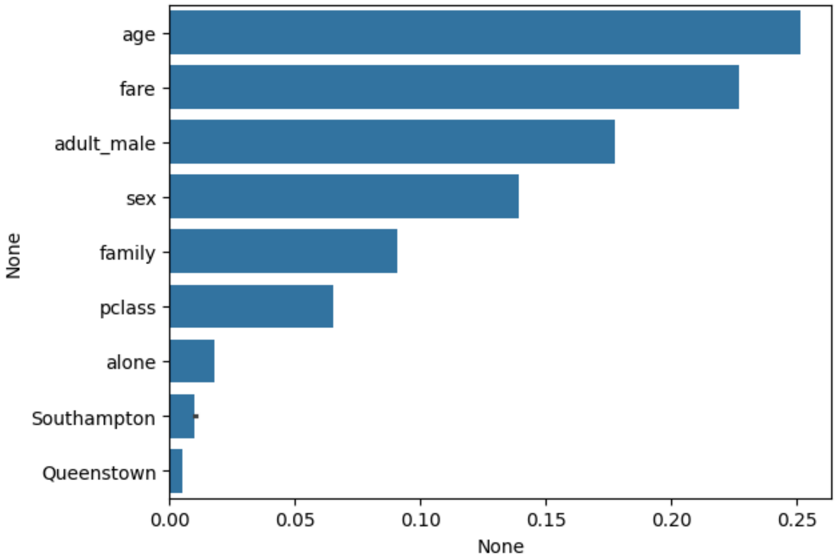
```
[0.06517924 0.1393711  0.25141982 0.22717674 0.17736677 0.01788452
 0.00519594 0.01027824 0.00528282 0.00971971 0.09112509]
```

```
vars_imp = pd.Series(rf.feature_importances_, index = X_train_up.columns).sort_values(ascending=False)
vars_imp
```

	0
age	0.251420
fare	0.227177
adult_male	0.177367
sex	0.139371
family	0.091125
pclass	0.065179
alone	0.017885
Southampton	0.010278
Southampton	0.009720
Queenstown	0.005283
Queenstown	0.005196

dtype: float64

```
sns.barplot(x=vars_imp, y=vars_imp.index)
plt.show()
```




```
vars_non_imp = ['Queenstown']

X_train_up = X_train_up.drop(vars_non_imp, axis=1)
X_val = X_val.drop(vars_non_imp, axis=1)
X_test = X_test.drop(vars_non_imp, axis=1)
```

```
# Régression logistique

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import GridSearchCV
```